VM Wrapping

Fake it till you make it

Johannes Henning[1], Tim Felgentreff[1] and Robert Hirschfeld[1]

[1]Hasso-Plattner-Institute, University of Potsdam, Germany; {first.last}@hpi.uni-potsdam.de

Abstract

Building or extending virtual machines to investigate new language features or optimization techniques presents several challenges. The overhead for implementing a new research VM for an existing language is immense and generally outweighs the parts interesting for the research question significantly. We propose a different approach for implementing research VMs that allows for lower implementation overhead and thus produce results quicker.

Categories and Subject Descriptors A1 [*Virtual Machines*]: []

Keywords Virtual Machine Implementation

1. Introduction

Many modern programming languages are executed inside an interpreter or *Virtual Machine* (VM). For popular programming languages such as Python or Java a multitude of VMs is available, offering different benefits or targeting different usage scenarios.
VM-design is an integral part of programming language research, especially optimization techniques implemented inside the VM e.g. *Just-In-Time Compiler* (JIT)-compilation and continue to be active research topics. In particular the RPython project [1] is researching optimization techniques which can be implemented language independently inside a VM-development framework. The RPython JIT compiler has been applied to several languages, yielding significant performance benefits.
However, doing such experiments is very time-intensive as altering an existing VM or implementing a new one requires significant implementation effort as well as knowledge about minute language internals. In particular, the research might have to spend a large amount of time implementing parts of the language that are not relevant for his/her research question, but necessary to support the targeted language. If this process of VM development could be streamlined further, it would be very beneficial to VM development research.
In this paper we propose an implementation approach for quickly generating meaningful measurements of the effectiveness of new optimization techniques or language features. This proposal centers on the idea of taking an existing VM and only overriding the research-relevant parts to quickly gain insights into the research question, while maintaining complete compatibility with the existing language implementation.
In particular the expensive implementation experiments we aim to simplify are:

1. Implementing new bytecodes or language features
2. Modifying internal VM objects and storage strategies
3. Implementing and testing new optimization techniques

We will outline background and related work, before presenting our proposed implementation and possible usage scenarios, after which we will conclude this position paper.

2. Background and Motivation

2.1 Virtual Machines and Interpreters

In the context of programming language development an interpreter can be considered equivalent to a VM as the differentiation is not relevant in practice but rather reflects conceptual differences. For the remainder of this paper we consider the two to be synonymous.
Interpreters are the implementation technique of choice for many modern programming languages, particularly dynamically-typed languages, as the alternative would be a compiler, which are inherently harder to write without having information about types at compile-time. Generally speaking interpreted languages are considered to be slower than compiled languages, while being more flexible and on a higher abstraction level. The two are also not mutually exclusive, as many interpreted languages are compiled to an intermediate representation before being interpreted, e.g. Java or Python.
We will focus on languages, like the aforementioned, that are implemented by byte-code interpreters. The most interesting part of a byte-code interpreter is its *Main-Interpreter Loop*

[Copyright notice will appear here once 'preprint' option is removed.]

(MIL), where the interpreter dispatches each byte-code to its implementation. This architecture is particularly interesting for our use-case, as it is very easy to add or override a single byte-code implementation, without altering anything else of the existing implementation.

2.2 RPython

RPython is a language implementation framework written in a subset of Python that can be compiled by being statically typed through type-inference during compile time. The most prominent VM written in RPython is the Python interpreter PyPy, a mature and compliant interpreter that outperforms the standard C-Python interpreter in many benchmarks [7].
RPython relies on a meta-tracing JIT compiler for optimizations at runtime.

2.2.1 Meta-Tracing JIT Compilation

A meta-tracing JIT compiler is an implementation technique for generating language independent JIT compilers, where the user program is traced indirectly by tracing the interpreter running the program. Ideally the meta-tracing JIT is able to perform all optimizationes a regular JIT comipler would be able to do.
needs work
JIT optimization techniques work by monitoring the running application, identifying hot code paths in it and compiling these paths with optimizations based on the runtime behavior of the application. This is particularly useful for dynamically-typed programming languages because while we do not know the type of a variable at compile time, the JIT can observe its type at runtime and monitor wether it ever changes. If it concludes that a variable is unlikely to change its type, it can replace several type-checks in the JIT compiled code with one single guard. Similarly, the JIT can observe whether a computed value is constant or an allocated object is unused in the remaining program and optimize comuptation and allocation away. These optimization techniques, i.e. constants folding, elidable function and allocation removal, have been discussed extensively in publications by members of the RPython team cite [3].
It is important to note that the functionality of the JIT compiler is part of RPython and can therefore be utilized by any VM implemented in it with minimal effort by adding a couple of *hints* to the MIL [4].

2.3 SQPyte

The SQPyte [5] prototype aimed at speeding up database access in PyPy by including SQLite in the runtime environment. By overriding selected byte-codes in SQLite's SQL engine it allowed the JIT compiler to optimize application and database operations at the same time. The prototype was build in a relatively short time period [2] and used a similar implementation technique to the one presented here.
Overall the project was able to produce insights into the optimization potential of an in-process database while maintaining complete compatibility with unmodified SQLite invocations and only modifying relevant parts of the code base. The SQPyte prototype did not modify any SQLite data-structures and did not need to be aware of specific database internals, while still allowing for JIT optimized SQL statement execution.

2.3.1 SQPyte as a Motivating Example

As demonstrated in the SQPyte project, an approach like VM wrapping can allow for experiments on established software projects, without incurring the massive implementation overhead of rebuilding the existing implementation in the research framework. More concretely: The general approach for this type of experiment would have required a reimplementation of a SQLite equivalent database inside the RPython framework or Python itself. Unless the newly implemented database would be able to support all SQLite features, it still would need to argue the applicability of the results to SQLite.
SQLite has been developed for several decades[1] and rebuilding it or exchanging significant parts of the implementation is no small feat. Without the used approach, it would not have been possible to conduct the experiments in the three weeksit took to build the prototype.
tim fragen
We believe the SQPyte project as a case study motivates VM wrapping as a general approach for implementing research prototypes for programming language experiments.

3. Proposed Implementation Strategy

Our approach aims at streamlining research VM development, simplifying implementations and overall reducing implementation time by relying on an existing implementation as much as possible. We propose to overwrite and extend the existing implementation and delegating everything that is not required by or interesting for our research question. This is done by *wrapping* the existing VM in our new VM and delegating where appropriate. Thereby we support the entire language, even if we have not implemented it ourselves. This is immensely helpful, because we can rely on existing benchmarks suits as well as an abundance of existing example applications.
There are several ways this delegation could be implemented, varying by implementation strategy and carrying different advantages and disadvantages, which we will discuss in the following section.
list

3.1 Granularity of Delegation

The first important question depends on the parts of the implementation that the researcher is interested in. For example, are we interested in implementing a new byte-code, changing the implementation of an existing one, using a different layout for internal VM objects or testing optimization techniques like unrolling?

[1] Development started 09/05/2000 https://www.sqlite.org/about.html

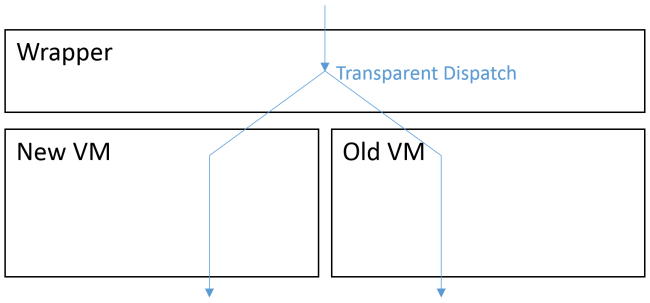


Figure 1: Conceptual architecture

The answer to this question is important for identifying the parts of the existing implementation that we need to extend or replace. A new byte-code could be added relatively easy to the MIL of the existing interpreter, while changing storage strategies for internal VM objects would require modifications spanning several bytecodes and rewriting the underlying data structures.The following sections present different delegation techniques appropriate for different granularities of delegation.

unrolling

3.1.1 Override existing Bytecodes

This is the most straightforward approach other than extending the existing VM altogether. The SQPyte Project discussed in section 2.3 used this approach. The idea is to compile the existing VM into our VM framework and run it as usual, only overriding byte-codes that we are interested in. On the plus side we retain all existing features of the VM, while the limitations of this approach lie in the existing implementation. As we can only redefine the execution of selected bytecodes, we cannot change the underlying data structures or make other major architectural changes. We also might require somewhat extensive knowledge of the existing implementation.

3.1.2 Transfer State, Dispatch Per Bytecode

By mapping our internal VM objects onto the wrapped VM's internal objects it would be possible to to switch bytecode execution between both VMs dynamically and transfer the internal VM state before each switch. This would allow for complete freedom in VM architecture, but would require a lot of knowledge about the existing VM in addition to runtime overhead for each switch.

An alternative way to ensure the same state in both VMs would be to execute both in lock-step, e.g. synchronizing after each bytecode. This would not be useful for performance benchmarks, but could help debugging and testing computational correctness.

expand?

3.1.3 Dispatch Per Function

Wrapped VM implementations could also be called on a per-function basis. Depending on the level of function 'pureness' the passing of relevant internal objects could be relatively simple. While this would require the VM to implement all basic language features, it would allow for complete implementation freedom, while allowing the developer to delegate heavily to the wrapped VM similarly to invoking a library.

3.1.4 Prescan, Dispatch Per Program

A different approach would be to keep both VMs separate from each other and select the appropriate VM through static analysis. Say our research VM only implements a subset of the specified language, we could determine wether the program uses language features outside of our implementation and transparently execute it in the wrapped VM. This approach would only make sense if the subset of the language we are implementing is large enough to write meaningful programs in. The advantages would be complete architectural freedom and requiring very little knowledge about the wrapped VM.

eval

3.2 Combining Abstraction Levels

The implementation approaches presented above can also work in conjunction with each other. For example, we might select the appropriate VM by analyzing byte- or program-code, but discover at runtime that the program uses an unimplemented language feature through a function like *eval*. At this point we could fall back on delegating to the wrapped VM via our *per function* or *per bytecode* approaches.

4. Usage Scenarios

We believe that the presented approach could be useful for multiple use-cases, presented below.

4.1 Research Prototypes

As discussed in section 2.3 the overhead of implementing equivalent technologies to create meaningful and applicable research results is a general problem of language implementation research. While RPython already makes it possible to experiment with JIT technologies with minimal overhead, it still requires the complete language to be implemented for the VM to be of practical use. While there have been some successful implementations of language interpreters in RPython, i.e. PyPy and RSqueak[2], there have been more research VMs that are unlikely to ever support the entire language they were targeted to implement, simply because its not connected closely enough to the research question the partial VM was build for.
We believe that VM wrapping can the a applicable solution for many similar problems in research. It is a way to get answers faster, much in the same way that RPython is. While

[2] https://github.com/HPI-SWA-Lab/RSqueak

the JIT optimizations RPython provides are not as sophisticated as what is available in specifically tailored VMs like Java hotspot or LuaJIT, the cost-benefit of RPython is far better. Similarly, the result of VM wrapping will not be an optimal VM for the targeted language, but a low-cost way of answering research questions about the language, while generating more meaningful results than alternative low-cost approaches.

4.2 Legacy Language Maintenance

There is another use-case for transparent dispatch between different VM implementations. We have observed problems for vendors of proprietary languages when deprecating language features. Since the vendor is not aware of all existing customer applications and features used within them, the vendor is often hesitant to deprecate at all, fearing suboptimal usability or lack of backwards compatibility. We believe that an approach like prescan (section **??**) could help to update such languages while maintaining backwards compatibility for overhauled language features. Such an approach could also prove useful for transition periods and continuous integration development, as parts of the language could be redesigned one at a time, while continuously providing a compatible and runable VM.

4.3 General Language VM Development

Also for the design of a new production VM for non-research purposes, VM wrapping could be a useful way of getting early feedback during development by allowing to execute programs even if important parts of the new VM have not been implemented yet.

5. Related Work

We are not aware of previous work describing the implementation strategies presented here, there are however two solutions to similar problems, nameley the already mentioned SQPyte project and UUOs used in MacLisp.

5.1 Unimplemented User Operations

Unimplemented User Operations (UUO) were used in MacLisp to dispatch op-codes to different implementations [6]. These unimplemented op-codes would trap the execution and allow calling user-defined functions. In practice this was used to switch transparently between compiled and interpreted versions of the VM and thus was not intended to dispatch between different VMs. However, this approach demonstrates the possibility of replacing byte-code implementations transparently at a lower abstraction level.

5.2 SQPyte

While our approach is partially identical to the work done by the SQPyte project, discussed in section 2.3, it was focused simply on getting results in the fastest and easiest way available and was not focused on the general applicability of the used implementation approach, but presents a valuable case study for our proposed implementation approach. In section 3 we presented several considerations and implementation addendums that were outside the scope of the SQPyte project and thus not discussed previously.

6. Future Work & Conclusion

We have presented an alternative approach to research VM development, which allows for streamlining parts of the process, by focusing on the research-relevant parts of the VM implementation and leaving the rest to an existing implemenation while maintaining compatibility with existing benchmarks suites and example applications. While VM wrapping seems feasible in theory, we have yet to test our different implementation approaches in practice, which is the logical next step and future work.

References

[1] Davide Ancona, Massimo Ancona, Antonio Cuni, and Nicholas D Matsakis. Rpython: a step towards reconciling dynamically and statically typed oo languages. In *Proceedings of the 2007 symposium on Dynamic languages*, pages 53–64. ACM, 2007.

[2] Carl Friedrich Bolz. personal communication. 2016.

[3] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijakowski, Michael Leuschel, Samuele Pedroni, and Armin Rigo. Allocation removal by partial evaluation in a tracing JIT. In *Proceedings of the 20th ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation*, PEPM '11, pages 43–52, New York, NY, USA, 2011. ACM.

[4] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijalkowski, and Armin Rigo. Tracing the meta-level: PyPy's tracing JIT compiler. In *Proceedings of the 4th workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems*, pages 18–25. ACM, 2009.

[5] Carl Friedrich Bolz, Darya Kurilova, and Laurence Tratt. Making an embedded dbms jit-friendly (artifact). In *DARTS-Dagstuhl Artifacts Series*, volume 2. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2016.

[6] Richard P Gabriel. *Performance and evaluation of LISP systems*, volume 263. MIT press Cambridge, Mass., 1985.

[7] PyPy Project. Pypy speed center, 2017. Accessed: 2017-03-31.